PhD Thesis Defense in Computer Science

Scheduler Guided OpenMP Execution in Cloud VMs

(Using Para-Virtualized Task Scheduling Insights For Barrier Synchronization)

17 December 2025 @ Inria, Paris

Student:
Himadri CHHAYA-SHAILESH

Advisors:
Dr. Julia LAWALL
Dr. Jean-Pierre LOZI

Reviewers:
Prof. Daniel HAGIMONT
Prof. François TRAHAY

Comité de suivi:
Dr. Nikolaos GEORGANTAS
Dr. Fabien HERMENIER

Examiners:
Prof. Pierre SENS
Prof. Noel De Palma
Prof. Gaël Thomas

Outline

Context

The Triad

Inria

1 Parallelization

- Modern computers have multiple CPUs.
- Applications can be parallelized using multiple threads that run on multiple CPUs concurrently.

The Triad

Inria

2 Scheduling

- The operating system (OS) manages hardware resources.
- The OS scheduler decides which thread runs on which CPU and for how long.

The Triad

Inria

3 Virtualization

- A physical machine (host) can run multiple virtual machines (VMs) using a hypervisor.
- Such a VM (guest) runs its own OS with its own scheduler.

The Triad

Inria

1. **Parallelization**
   → Using GCC's implementation of OpenMP (`libgomp`).
2. **Scheduling**
   → Using Linux's Earliest Eligible Virtual Deadline First Scheduler (`EEVDF`).
3. **Virtualization**
   → Using the Quick EMUlator (`QEMU`) and the Kernel-based Virtual Machine (`KVM`).

OpenMP

Parallelism Using OpenMP

A simple sequential loop in C

```
1 for (i = 0; i < N; i++)
2     work();
```

A simple loop in C parallelized using OpenMP

```
1 #pragma omp parallel for
2 for (i = 0; i < N; i++)
3     work();
```

What is OpenMP? [1]

- An Application Program Interface (API) useful for achieving multi-threaded, shared memory parallelism.
- It provides:
  1. Compiler Directives,
  2. Runtime Library Routines,
  3. Environment Variables.
- It is implemented as a runtime library by the compiler (e.g., `libgomp` in GCC)
- The programmer has the full control over parallelization.

[1] https://hpc-tutorials.llnl.gov/openmp

In the previous example, OpenMP took care of...

Inria

- Determining how many workers to use for parallelization i.e. the Degree of Parallelism (DoP).
- Creating the worker threads using the `pthread` library
- Distributing the loop iterations evenly among the workers.
- Launching the workers to execute the loop iterations concurrently.
- Synchronizing the workers at the end of the loop.
- Terminating the workers.

OpenMP 101: Threads

Inria

- A fork-join model, where a master thread and several worker threads form a team to execute parallel work.
- The value of DoP is resolved at the beginning of a parallel region.
- OpenMP creates, re-uses, and terminates threads as and when needed.

OpenMP 101: Barriers

Inria

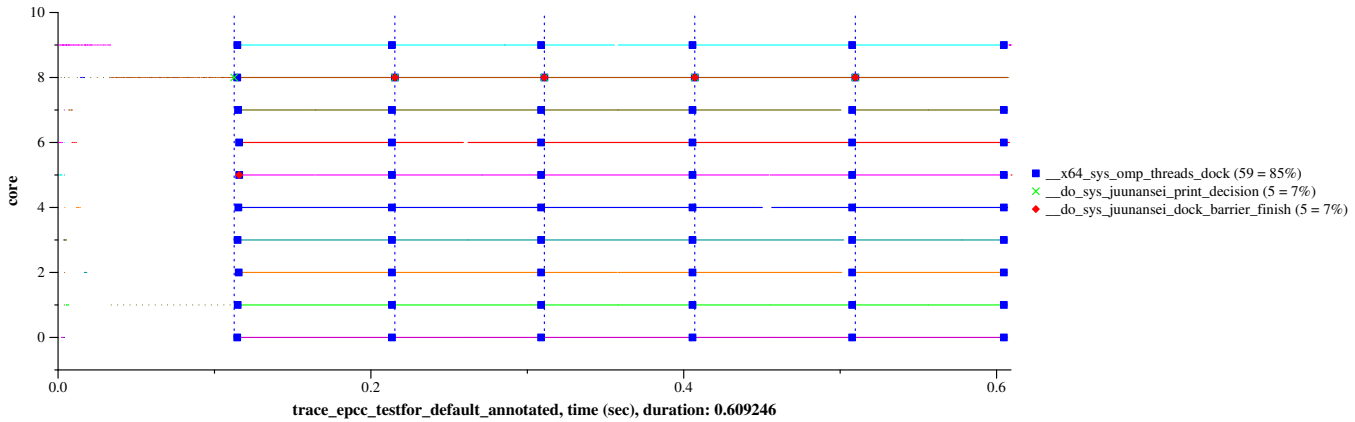
- A barrier marks a point where all worker threads must finish their work before any can proceed.
- Some threads finish earlier than others, so early arrivals at the barrier must wait for the stragglers.
- Threads can wait at a barrier either by spinning or by blocking.
- Barrier usage in OpenMP:
  - Inter-region barriers (ThreadsDock), which marks the beginning of a new parallel region.
  - Intra-region barriers (TeamBarrier), which synchronizes threads within a parallel region.

Parallelism Using OpenMP

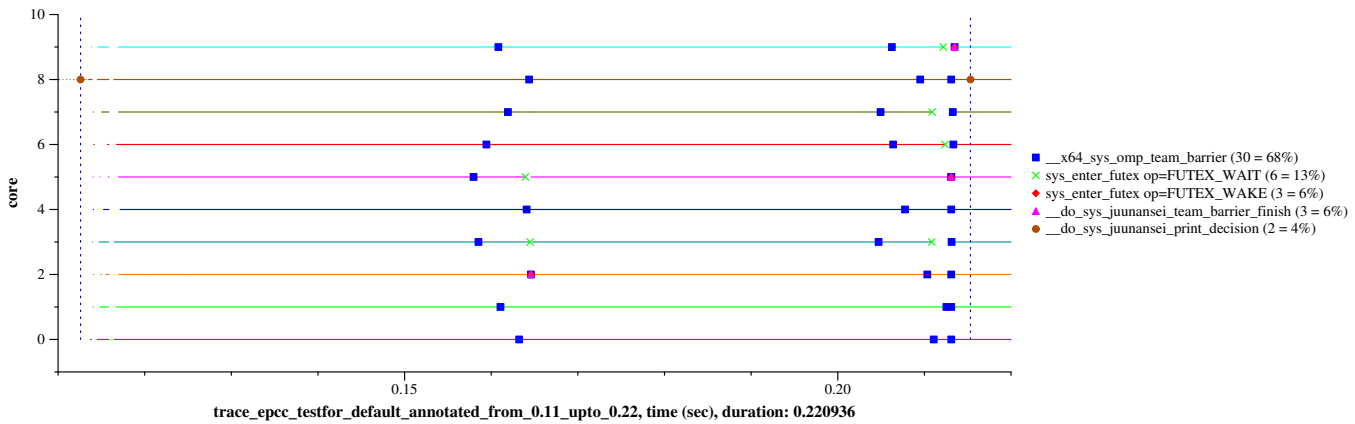
A simplified version of the `testfor()` from EPCC

```
1  void testfor(void) {
2      int i, j, k;
3      /* Five parallel regions */
4      for (k = 0; k < 5; k++) {
5          #pragma omp parallel private(j)
6          {
7              /* Two parallel loops per parallel region */
8              for (j = 0; j < 2; j++) {
9                  #pragma omp for
10                 for (i = 0; i < 1000; i++)
11                     delay(250);
12             }
13         }
14         delay(1000);
15     }
16 }
```

Barrier Visualization



Five parallel regions of the `testfor()` example



Single parallel region of the `testfor()` example

Default choices in libgomp

Static choices that applies to all parallel regions and barriers:

1. DoP = Number of online CPUs in the system.
   (can be overridden by setting the `OMP_NUM_THREADS`.)
2. Waiting at barriers: Spin then block.
   (can be overridden by setting the `OMP_WAIT_POLICY`.)

Scheduling

Task Scheduling in Linux

Inria

- A task (thread/process) is a runtime execution entity.
- A task goes through various states during its lifetime, e.g. running, sleeping, waking up, interrupted, etc.

Task Scheduling in Linux

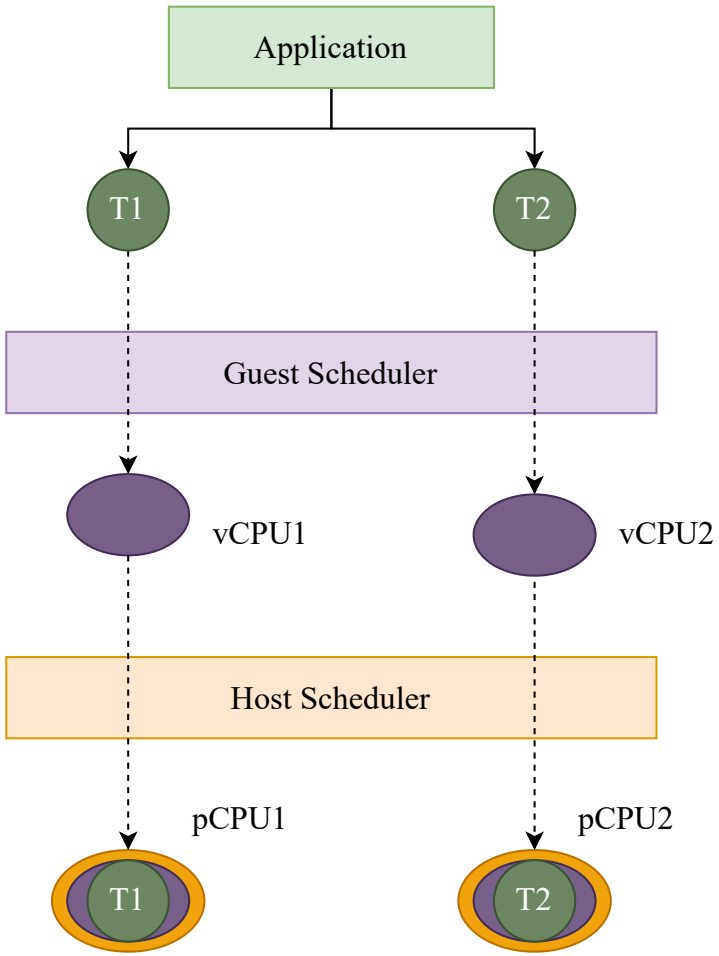
Inria

- A task (thread/process) is a runtime execution entity.
- A task goes through various states during its lifetime, e.g. running, sleeping, waking up, interrupted, etc.
- The **scheduler** is responsible for managing the tasks.
- Linux uses a fair scheduler, which strives to distribute tasks fairly across all CPUs.

Scheduling 101

- The `task_struct` data structure represents a task using numerous fields.
  e.g. `pid`, `comm`, `state`, `priority`, `policy`, etc.
- A runqueue is a per-CPU data structure that holds all tasks eligible to run on that CPU.
- Stopping a task from running is called preemption.
- Switching between two tasks is called a context switch.

Dual level of task scheduling in the cloud



Life of a vCPU

Inria

- A vCPU is created when a VM is launched.
- A vCPU is destroyed when the VM is terminated.
- The host scheduler treats vCPUs like any other regular task.

Life of a vCPU

- A vCPU is created when a VM is launched.
- A vCPU is destroyed when the VM is terminated.
- The host scheduler treats vCPUs like any other regular task.
- **A vCPU can be preempted at any time on the host**

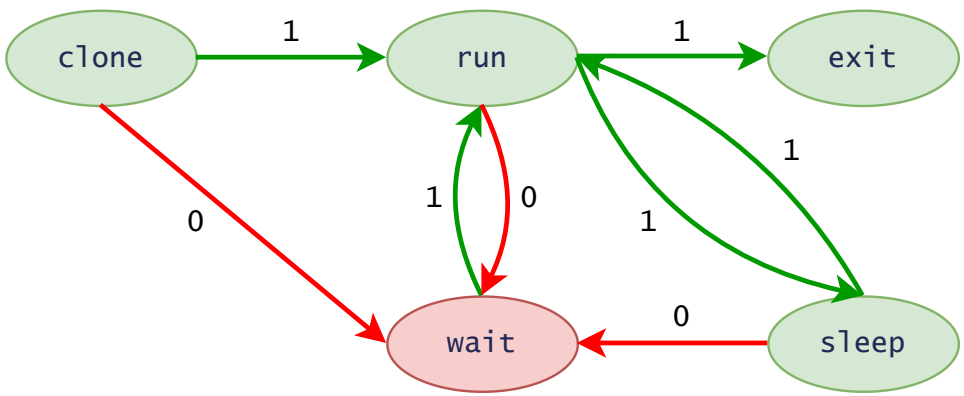


Figure 1: vCPU state transitions on the host

Consequences of ill-timed vCPU preemptions

Inria

- Lock holder preemption problem
- Lock waiter preemption problem
- Blocked-waiter wake up problem
- Readers preemption problem
- RCU reader preemption problem
- Interrupt context preemption problem

[1] Nicely described with existing solutions in "Scaling Guest OS Critical Sections with eCS" by S. Kashyap et al.

Thesis

Motivation

- OpenMP execution in cloud VMs is commonplace.
- Oversubscription is a popular cost-cutting practice in cloud deployments.
- OpenMP execution heavily relies on barrier synchronization.
- Barrier synchronization in an oversubscribed virtualized context is not well studied.

Opportunity

- No prior work has explored adding virtualization-aware task scheduling to barrier synchronization within VMs.

Opportunity

- No prior work has explored adding virtualization-aware task scheduling to barrier synchronization within VMs.
- Our prior experience with the scheduler, along with our in-house visualization tools, equips us to undertake this exploration.

Opportunity

- No prior work has explored adding virtualization-aware task scheduling to barrier synchronization within VMs.
- Our prior experience with the scheduler, along with our in-house visualization tools, equips us to undertake this exploration.
- It is a timely exploration amid ongoing efforts to standardize the para-virtualized task scheduling interface in the upstream Linux kernel.

Thesis

Scheduler Guided OpenMP Execution in Cloud VMs:

- Combine task-scheduling insights from both the schedulers.
- Guide barrier-heavy OpenMP runtime decisions.
- Improve performance inside oversubscribed Linux VMs.

Contributions

1. **Phantom Tracker**:
   An algorithm that tracks vCPU states during OpenMP execution in VMs.

Contributions

1. **Phantom Tracker**:
   An algorithm that tracks vCPU states during OpenMP execution in VMs.
2. **pv-barrier-sync**:
   - A paravirtualized barrier synchronization mechanism.
   - Dynamically switches between spinning and blocking on a per-thread and per-barrier basis.

Contributions

Inria

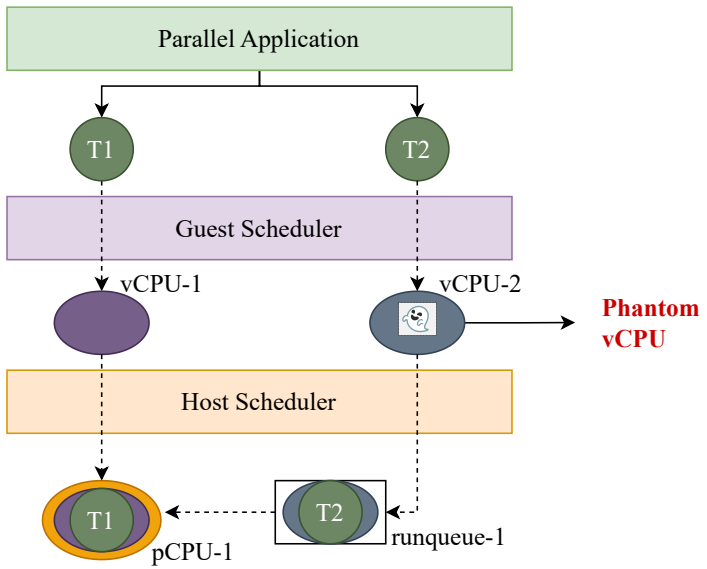
1. **Phantom Tracker**:
   An algorithm that tracks vCPU states during OpenMP execution in VMs.
2. **pv-barrier-sync**:
   - A paravirtualized barrier synchronization mechanism.
   - Dynamically switches between spinning and blocking on a per-thread and per-barrier basis.
3. **Juunansei**:
   An OpenMP runtime extension for improving the performance inside oversubscribed virtualized environments.

Phantom Tracker

Phantom vs. viable vCPUs under oversubscription

A vCPU is a phantom if:

1. It is currently waiting in the runqueue of a pCPU on the host.
2. A guest task that was running on this vCPU is now stalled because the vCPU is not executing.

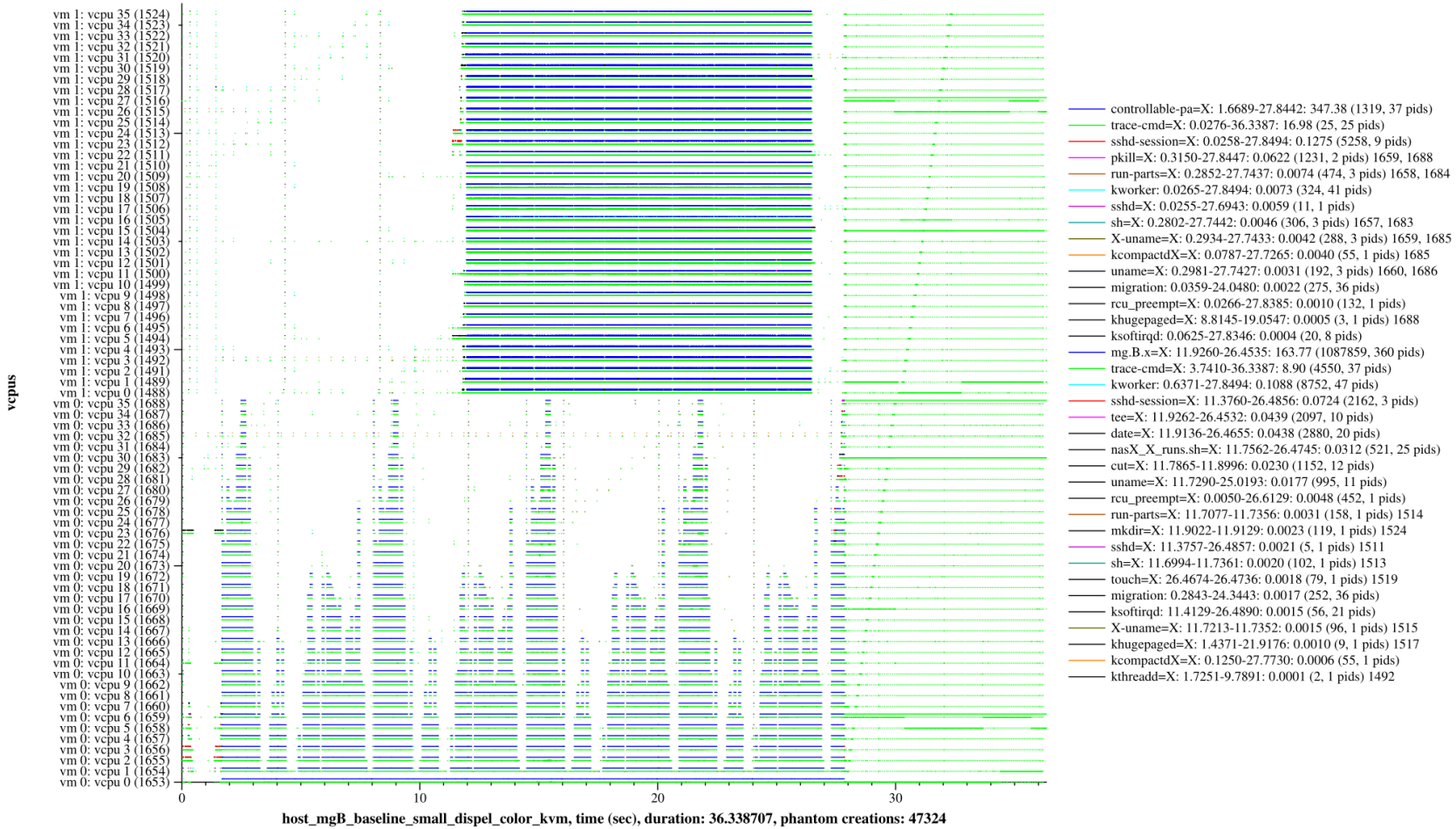


Phantom vCPU Visualization

Inria

We can visualize phantom vCPUs using the *tracing* facility in the Linux kernel and our in-house **schedgraph** visualization tools.

Phantom vCPUs in the real world



Experiment visualization: mg.B.x in target VM1 + noise in competing VM2

Phantom vCPUs in the real world

Closer look: Phantoms for target VM1

Phantom Tracker Overview

Inria

1. Enable communication between the host and guest schedulers.

Phantom Tracker Overview

Inria

1. Enable communication between the host and guest schedulers.
2. Register OpenMP threads in the guest Scheduler.
3. Monitor guest scheduler's decisions for the registered threads.

Phantom Tracker Overview

Inria

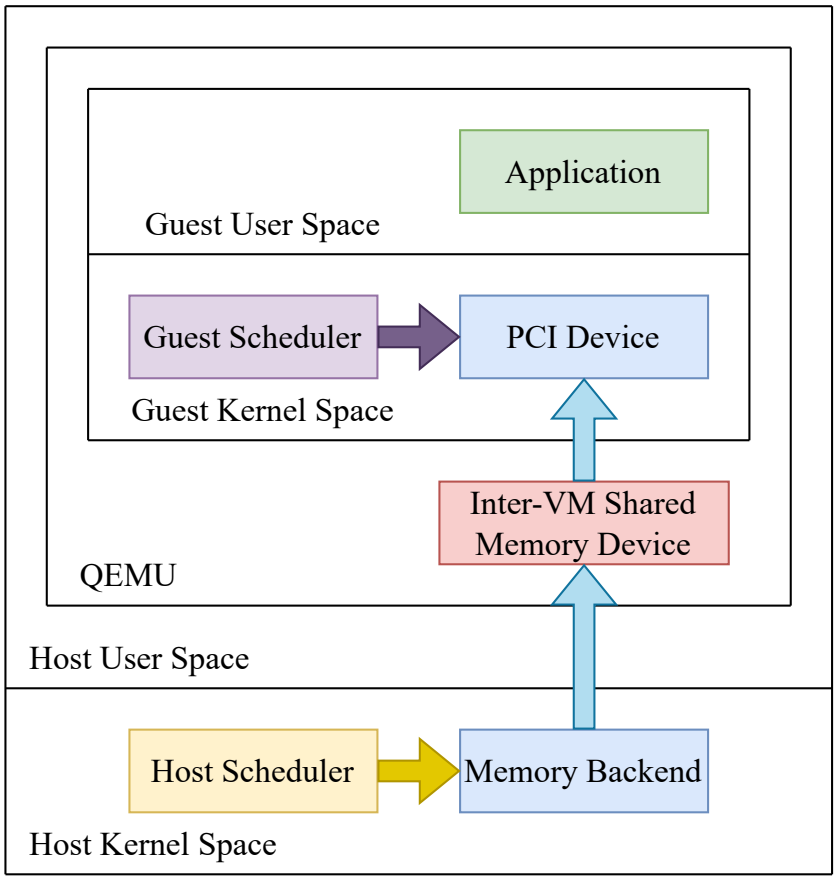
1. Enable communication between the host and guest schedulers.
2. Register OpenMP threads in the guest Scheduler.
3. Monitor guest scheduler's decisions for the registered threads.
4. Register target VM's vCPUs in the host Scheduler.
5. Monitor host scheduler's decisions for the registered vCPUs and keep track of phantom vCPU occurrences.

Phantom Tracker Overview

Inria

1. Enable communication between the host and guest schedulers.
2. Register OpenMP threads in the guest Scheduler.
3. Monitor guest scheduler's decisions for the registered threads.
4. Register target VM's vCPUs in the host Scheduler.
5. Monitor host scheduler's decisions for the registered vCPUs and keep track of phantom vCPU occurrences.
6. Compute `phantom_average` on the host.

Communication Between Host and Guest Schedulers



Communication Between Host and Guest Schedulers

Inria

```
1 struct hg_message {
2   u64 msg;
3   u64 time;
4   u64 padding[6];
5 } __attribute__ ((packed, aligned(64)));
```

Registering OpenMP Threads in the Guest Scheduler

Inria

```
sysctl -w kernel.phantom_tracker_register=benchmark
```

Registering OpenMP Threads in the Guest Scheduler

Inria

```
File: include/linux/phantom_tracker.h
- - -
. . .
#define TRACKING_COMM_LEN 16 /* same as TASK_COMM_LEN */
extern char tracking_comm[TRACKING_COMM_LEN];
. . .

File: kernel/sched/core.c
- - -
. . .
static const struct ctl_table sched_core_sysctls[] = {
  . . .
  {
    .procname       = "phantom_tracker_register",
    .data           = &tracking_comm,
    .maxlen         = TRACKING_COMM_LEN,
    .mode           = 0644,
    .proc_handler   = proc_dostring,
  },
};
. . .
```

Monitoring in the Guest Scheduler

Inria

We monitor the context switches inside the guest:

- We use one `hg_msg` entry per vCPU.
- When a registered thread gets scheduled on vCPU-i:
  → `hg_msg[i].msg = 1;`
- When a registered thread gets scheduled out on vCPU-i:
  → `hg_msg[i].msg = 0;`

Registering Target VM's vCPUs in the Host Scheduler

Inria

- Each VM is launched inside a cgroup.
- Target VM loads the shared memory device.

Registering Target VM's vCPUs in the Host Scheduler

Inria

We keep track of the vCPUs of the target VM by:

- Adding an extra field `vcpu_id` in the `struct task_struct`.
- We initialize this field in `kvm_vcpu_init()`.

Monitoring & Counting Phantoms in the Host Scheduler

Inria

Algorithm: Recording samples in `ttwu_do_wakeup()`

```
1: procedure ttwu_do_wakeup(task)
2:     if is_vcpu(task) and not is_state_running(task)
        and not is_phantom(task) then
3:         nr_phantoms = atomic_inc_return(phantom_count)
4:         timestamp = ktime_to_us()
5:         record_sample(nr_phantoms, timestamp)
6:     end if
7: end procedure
```

Monitoring & Counting Phantoms in the Host Scheduler

Algorithm: Recording samples in `__schedule()`

```
 1: procedure __schedule(())
 2:     if is_vcpu(prev) and
         is_state_running_or_waking(prev) and not is_phantom(prev) then
 3:         vmid = get_vmid(prev)
 4:         nr_phantoms = atomic_inc_return(phantom_count)
 5:         record_sample = true
 6:     else if is_vcpu(next) andis_phantom(next) then
 7:         vmid = get_vmid(next)
 8:         if is_vcpu(prev) and
              vmid == get_vmid(prev) then
 9:             skip
10:         else
11:             nr_phantoms = atomic_dec_return(phantom_count)
12:             record_phantom_sample = true
13:         end if
14:     end if
15: end procedure
```

Computing `phantom_average` on the Host

- Two circular buffers: *collection* and *processing*
- A global index for recording samples, which is incremented atomically on each sample recorded.

Computing `phantom_average` on the Host

Inria

- Two circular buffers: *collection* and *processing*
- A global index for recording samples, which is incremented atomically on each sample recorded.
- Current prototype uses `MAX_SAMPLES = 4096`
- Samples recorded during the interval of a scheduler tick are processed at the end of the tick.

Computing `phantom_average` on the Host

Inria

- Two circular buffers: *collection* and *processing*
- A global index for recording samples, which is incremented atomically on each sample recorded.
- Current prototype uses `MAX_SAMPLES = 4096`
- Samples recorded during the interval of a scheduler tick are processed at the end of the tick.
- Edge cases:
  - Out of order samples are ignored.
  - Extra boundary samples:
    ⟨nr_phantoms_tick_start, tick_start_time⟩
    ⟨nr_phantoms_tick_end, tick_end_time⟩

Computing phantom_average on the Host

Inria

Algorithm: Processing phantom_count samples in sched_tick()

```
1:  procedure sched_tick(())
2:      swap(collection, processing)
3:      for i = first_recorded_sample; i < last_recorded_sample; i++ do
4:          if i == first_recorded_sample then
5:              value = get_sample_value(i)
6:              duration =
                        get_sample_timestamp(i) - tick_start_time
7:          else if i == last_recorded_sample then
8:              value = get_sample_value(i)
9:              duration =
                        tick_end_time - get_sample_timestamp(i)
10:         else
11:             value = get_sample_value(i)
12:             duration =
                        get_sample_timestamp(i+1) - get_sample_timestamp(i)
13:         end if
14:         total_duration += duration
15:         sum += value * duration
16:     end for
17:     phantom_average = sum / total_duration
18:     write_to_ivshmem(phantom_average)
19: end procedure
```

Pv-barrier-sync

To Spin or Not to Spin

Huang et al.'s finding reveal that for barrier synchronization

- Using spinning can lead to cascading performance loss and a significant waste of CPU cycles under oversubscription.

[1] HPDC'21: "Towards Exploiting CPU Elasticity via Efficient Thread Oversubscription"

To Spin or Not to Spin

Huang et al.'s finding reveal that for barrier synchronization

- Using spinning can lead to cascading performance loss and a significant waste of CPU cycles under oversubscription.
- Using blocking is inefficient because Linux kernel's task sleep and wake-up process involves multiple queue locking and thread state transitions, making it too expensive under oversubscription.

[1] HPDC'21: "Towards Exploiting CPU Elasticity via Efficient Thread Oversubscription"

To Spin or Not to Spin

Kontothanassis et al. propose using scheduler information for making optimal choices between spinning and blocking at barriers.

- For N threads trying to get through the barrier using P CPUs where $N >= P$:

[1] PPOPP '93: "Using scheduler information to achieve optimal barrier synchronization performance"

To Spin or Not to Spin

Kontothanassis et al. propose using scheduler information for making optimal choices between spinning and blocking at barriers.

- For N threads trying to get through the barrier using P CPUs where $N >= P$:
- The optimal policy would force the first $N - P$ threads to block, so that the remaining P can finish their work.

[1] PPOPP '93: "Using scheduler information to achieve optimal barrier synchronization performance"

Overview

Pv-barrier-sync...

1. extends Kontothanassis et al.'s proposal to cloud VMs using paravirtualized scheduling insights generated by Phantom Tracker.

Overview

Pv-barrier-sync...

1. extends Kontothanassis et al.'s proposal to cloud VMs using paravirtualized scheduling insights generated by Phantom Tracker.
2. respects Huang et al.'s findings:
   - Maximizes spinning as long as there are no phantoms.
   - Minimizes blocking and use it as a last resort if phantoms are detected.

Phantom-Guided Spinning

- Per-barrier and per-thread decision:
  "Should I block at this barrier?"
- When phantoms are detected,
  we block as many worker threads as there are phantoms.

Implementation in libgomp

Inria

Algorithm

```
1: procedure do_spin
2:     loop
3:         if phantoms_detected then
4:             if atomic_sub(to_block, 1) > 0 then
5:                 block()
6:             end if
7:             if phantoms_increasing then
8:                 should_i_block = check_to_block_more()
9:                 if should_i_block then
10:                    block()
11:                end if
12:            end if
13:        else if itr_count % onems_spins == 0 then
14:            phantom_average = read_ivshmem()
15:            update_pv_barrier_sync_stats(phantom_average)
16:        end if
17:    end loop
18: end procedure
```

Juunansei

Overview

Juunansei...

- aims to maximize phantom-free execution of OpenMP inside the target VMs.
- implemented as an extension to libgomp.

Three part solution

Inria

1. Phantom Tracker guided DoP adaptation at the beginning of each new parallel region.
2. Pv-barrier-sync mechanism used for all ThreadsDock barriers and TeamBarriers.
3. A lightweight task-affinity mechanism instructs the guest scheduler to place the OpenMP worker threads on the first *DoP* vCPUs of the target VM.

Scheduler-guided DoP Adaptation

- Override `OMP_DYNAMIC` environment variable.
- Extend Phantom Tracker to estimate the number of available idle pCPUs on the host.
- Replaces the `loadavg` based DoP adaptation in libgomp with `phantom_average` and `idle_average` guided adaptation.

[1] Juunansei extends Phantom Tracker to produce per-tick `idle_average` on the host.

Host Watermark Production

At every scheduler tick, the host produces a 64-bit watermark:

```
0xTTTTTTTT PPPP IIII
```

Host Watermark Production

At every scheduler tick, the host produces a 64-bit watermark:

`0xTTTTTTTT PPPP IIII`

where:

- The lower 16 bits `IIII` represent the value of `idle_average`.
- The next 16 bits `PPPP` represent the value of `phantom_average`.
- the upper 32 bits `TTTTTTTT` store the host timestamp recorded at the end of the scheduler tick.

Bi-directional DoP Scaling in the Guest

The DoP resolution is handled by the master thread, which reads the latest host watermark when entering the `gomp_dynamic_max_threads()` function:

- `phantom_average` > 0
  $\rightarrow$ scale down DoP
- `phantom_average` $= 0$ and `idle_average` > 0
  $\rightarrow$ scale up DoP
- `phantom_average` $= 0$ and `idle_average` $= 0$
  $\rightarrow$ maintain current DoP

Stability requirement

In order to be conservative while scaling up the DoP:
Juunansei only consumes the minimum of reported `idle_average` values over the last `stability_requirement` ticks.

- Default value: `stability_requirement` = 2 ticks
- If phantoms are detected after scaling up the DoP,
  → `stability_requirement *= 2;`

Special Cases

Inria

1. First Decision
   → DoP = `min_idle_average` over last 2 ticks
2. Same Tick Decisions
   → reuse the last decision

Active DoP

Active DoP: Number of OpenMP threads currently running inside the target VM.

- At the beginning of each new parallel region
  $\rightarrow$ Active DoP = Adapted DoP
- In case of phantom detection during barrier synchronization
  $\rightarrow$ Active DoP = DoP - Number of detected phantoms

juunansei_cpuset

- At the beginning of each new parallel region:
  If $DoP = N$
  $\rightarrow$ the last thread to reach ThreadsDock applies
  `juunansei_cpuset` = [0, N-1] on the entire team.

juunansei_cpuset

Inria

- At the beginning of each new parallel region:
  If $DoP = N$
  $\rightarrow$ the last thread to reach ThreadsDock applies
  `juunansei_cpuset` = [0, N-1] on the entire team.
- In case of phantom detection with pv-barrier-sync:
  If P phantoms are detected at a barrier
  $\rightarrow$ one of the spinning threads adjusts
  `juunansei_cpuset` = [0, N-P-1] and updates the entire team.
  If more phantoms are detected during the same region
  $\rightarrow$ no further updates to `juunansei_cpuset`.

Juunansei in Action

Inria

The next figure refused to be compressed into a slide!

Evaluation

Evaluation Goals

- Quantify the performance improvements achieved by Juunansei in oversubscribed virtualized environments.
- Analyze the behavior of Juunansei under varying levels of resource contention caused by a competing VM running a random workload.
- We use three different experimental set-ups (Small, Medium, Large) to validate the scalability of Juunansei.
- Evaluate Juunansei using the NAS parallel benchmarks as representative OpenMP applications.
- We vary the load in the competing VM by using a random workload of varying intensity.

Experimental Setup - Small

Oversubscription ratios = 1:2

- Host: Intel Xeon Gold 5220, 36 pCPUs*, 92 GB RAM
- Guest-1 (Target VM): 36 vCPUs, 45 GB RAM
- Guest-2 (Competing VM): 36 vCPUs, 45 GB RAM

* Hyper-Threading enabled (2 threads per core)

Experimental Setup - Medium

Oversubscription ratios = 1:2

- Host: Intel Xeon Platinum 8358, 64 pCPUs*+, 500 GB RAM
- Guest-1 (Target VM): 64 vCPUs, 45 GB RAM
- Guest-2 (Competing VM): 64 vCPUs, 45 GB RAM

* Hyper-Threading enabled (2 threads per core)

+ Single socket; pCPUs of the other socket hot-unplugged

Experimental Setup - Large

Oversubscription ratios = 1:2

- Host: Intel Xeon Platinum 8568Y+, 96 pCPUs$^{*+\cdot}$, 500 GB RAM
- Guest-1 (Target VM): 96 vCPUs, 45 GB RAM
- Guest-2 (Competing VM): 96 vCPUs, 45 GB RAM

* Hyper-Threading enabled (2 threads per core)

+ Single socket; pCPUs of the other socket hot-unplugged

Experimental Setup - Software versions

- Host, Guest OS:
  Debian GNU/Linux Trixie
- Host, Guest Kernel:
  Linux v6.14
- Guest OpenMP runtime:
  `libgomp` from GCC branch-releases/gcc-14,
  commit-569f826774a
- Hypervisor:
  QEMU v10.0.2 with KVM provided by the host kernel

Target VM runs NAS parallel benchmarks

Table 1: Benchmark characteristics

Application	Input Size	Parallel Regions	Barriers
BT	Class B	1022	2234
CG	Class B	35	10123
FT	Class B	112	224
LU	Class B	516	2810
MG	Class B	1281	3071
SP	Class B	3616	7644
UA	Class B	38769	80503

Competing VM runs a random spinners workload

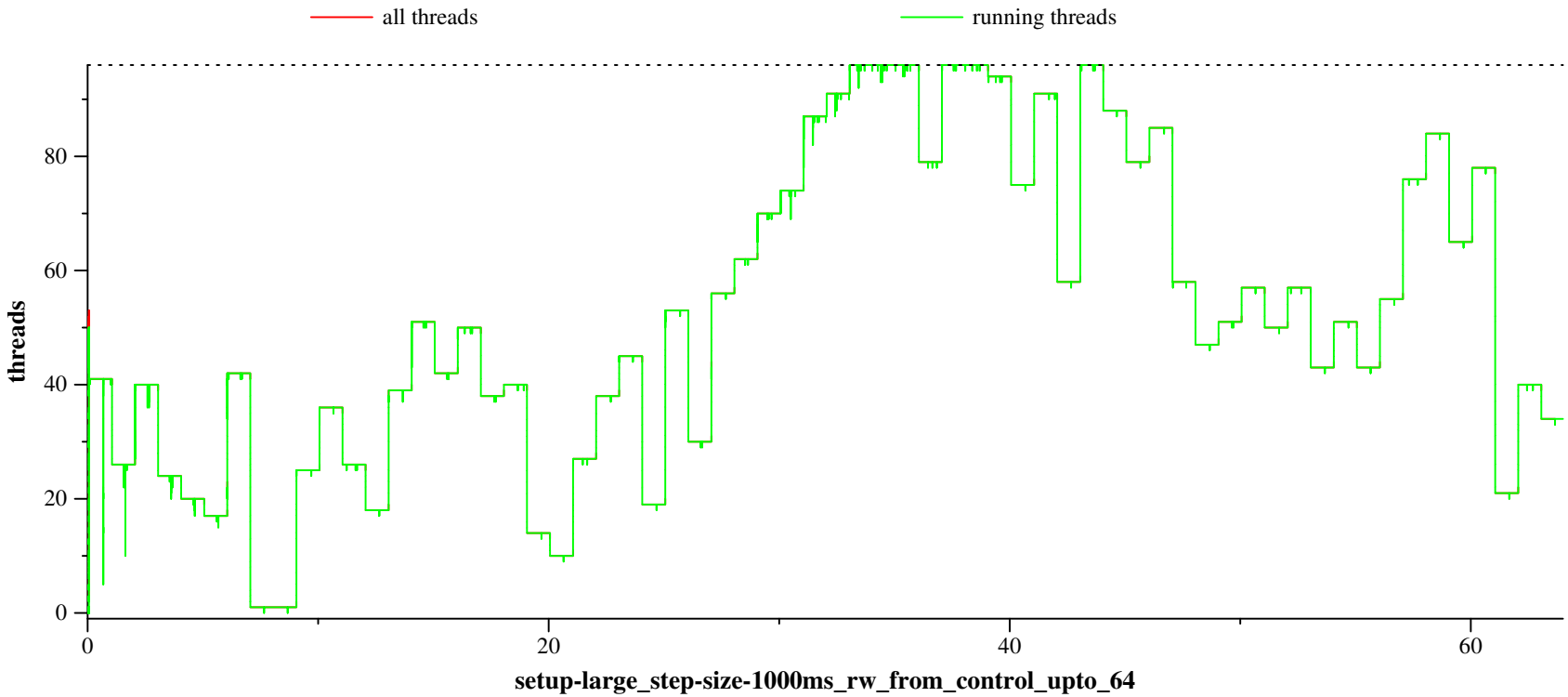


Figure 2: Execution inside the competing VM (Set-up: Large)

Random workload characteristics: Execution sequence

- Pseudorandom sequence (`exec_seq`) determines the number of threads executing at each step
- The first element of the sequence is chosen uniformly at random in the range [1, nr_vcpus]
- Each subsequent element is generated using a Gaussian (normal) distribution bounded between 1 and nr_vcpus.
- The Gaussian generator is centered around the previous sequence value (`exec_seq[i-1]`) for smooth transitions rather than abrupt jumps.

Random workload characteristics: Sleep sequence

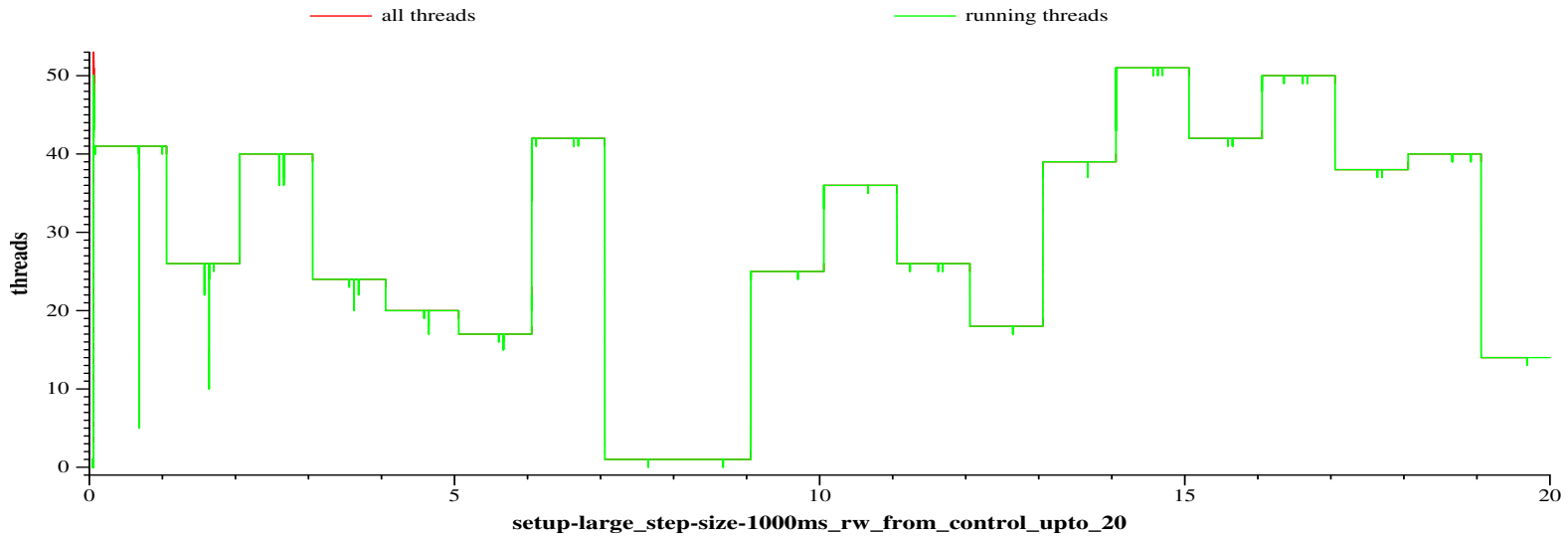
Inria

- Sleep schedule (`sleep_seq`) is derived from a previously generated execution sequence (`exec_seq`).
- Each element indicates how many steps a thread should sleep if it is not selected to run at the current step.
- Gives variability in sleep durations across threads and steps, simulating diverse workload patterns.

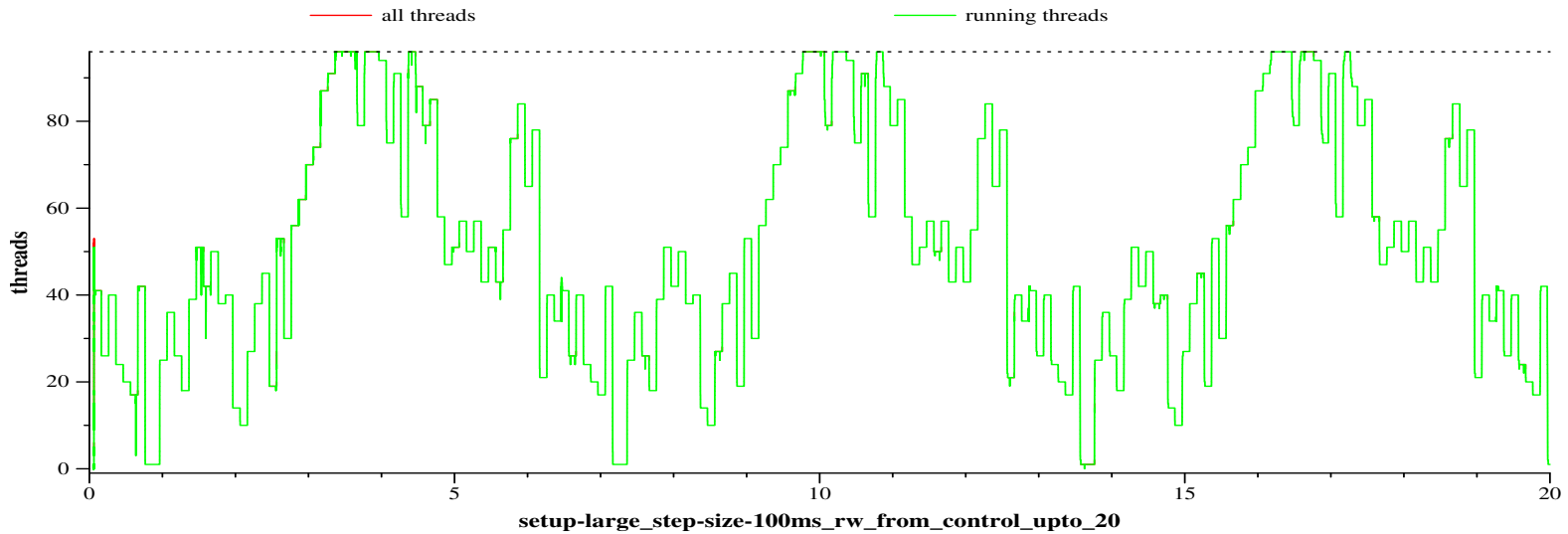
Random workload characteristics: Thread function

```
#define SEQ_LEN 64
void *thread_fn(void *arg) {
  ... /* Initialization */
  while (1) {
      ... /* Get current time */
      if (id < exec_seq[step]) {
        do {
          if (elapsed_ns >= step_size)
            break;
          } while (1);
          step = (step + 1) % SEQ_LEN;
      }
      else {
        t = sleep_seq[id][step] * (step_size / 1000);
        step = (step + sleep_seq[id][step]) % SEQ_LEN;
        usleep(t);
      }
  }
  ... /* Cleanup */
}
```

Random workload characteristics: Step Size



Step size = 1000 ms (Set-up: Large)



Step size = 100 ms (Set-up: Large)

Baseline vs. Juunansei set-up

- **Baseline**:
  - `OMP_NUM_THREADS` = Number of vCPUs of the target VM
  - `OMP_WAIT_POLICY` = passive
  - Host, Guest run vanilla kernels
  - NAS benchmarks load default `libgomp` from GCC branch-releases/gcc-14, commit-569f826774a

Baseline vs. Juunansei set-up

- **Baseline**:
  - `OMP_NUM_THREADS` = Number of vCPUs of the target VM
  - `OMP_WAIT_POLICY` = passive
  - Host, Guest run vanilla kernels
  - NAS benchmarks load default `libgomp` from GCC branch-releases/gcc-14, commit-569f826774a
- **Juunansei**:
  - `OMP_DYNAMIC` = true
  - Host, Guest run Linux kernels with Phantom Tracker patches
  - NAS benchmarks load `libgomp` with Juunansei patches

Performance metrics: Runtime

Method used to derive the runtime:

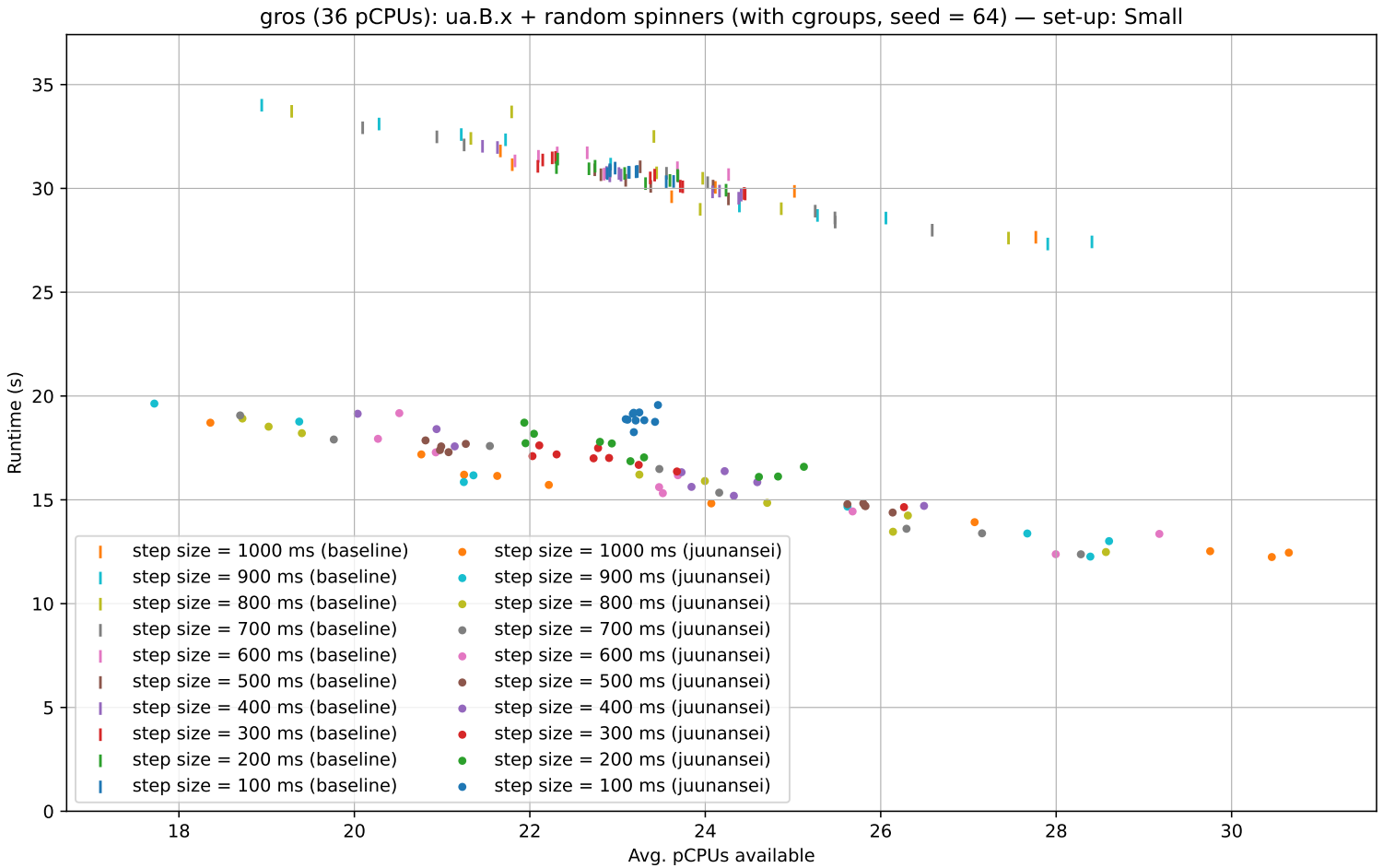
- `t_start` = kernel time when the first OpenMP worker thread is forked
- `t_end` = kernel time when the last OpenMP worker thread terminates
- `runtime` = `t_end - t_start` (reported in seconds)

Performance metrics: Available avg. pCPUs

Method used to derive the available avg. pCPUs:

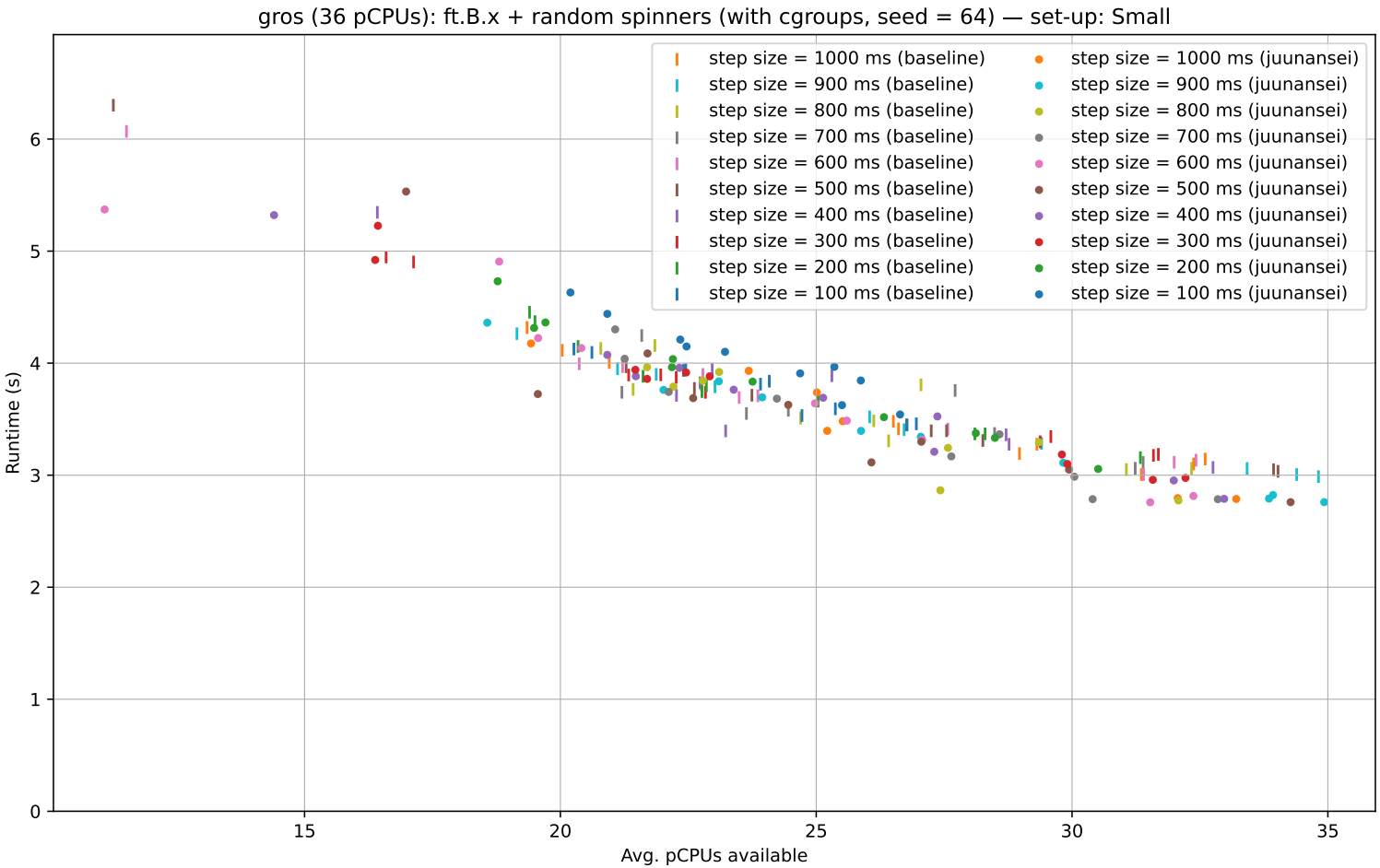
- Plot nr_running_vcpus vs. time curve for the vCPUs of the competing VM using the running_waiting tool
- Flip the curve and compute the area under the curve (`AUC`) during the period of [`t_start`, `t_end`] using the trapezoidal rule
- `available_avg_pcpus` = `AUC` / `runtime`

Results: Experiment set-up: Small [1/2]



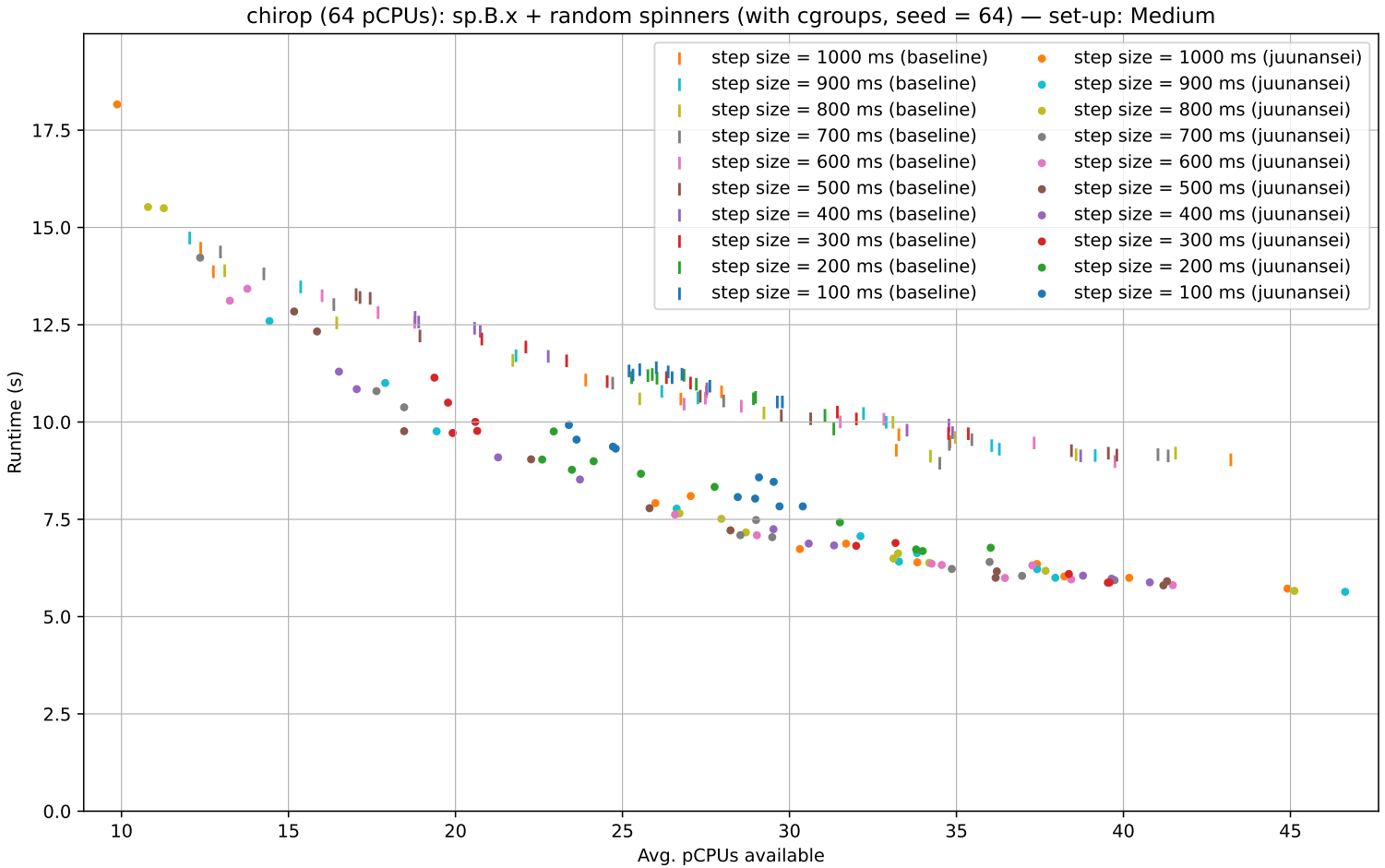
Target OpenMP application: UA

Results: Experiment set-up: Small [2/2]



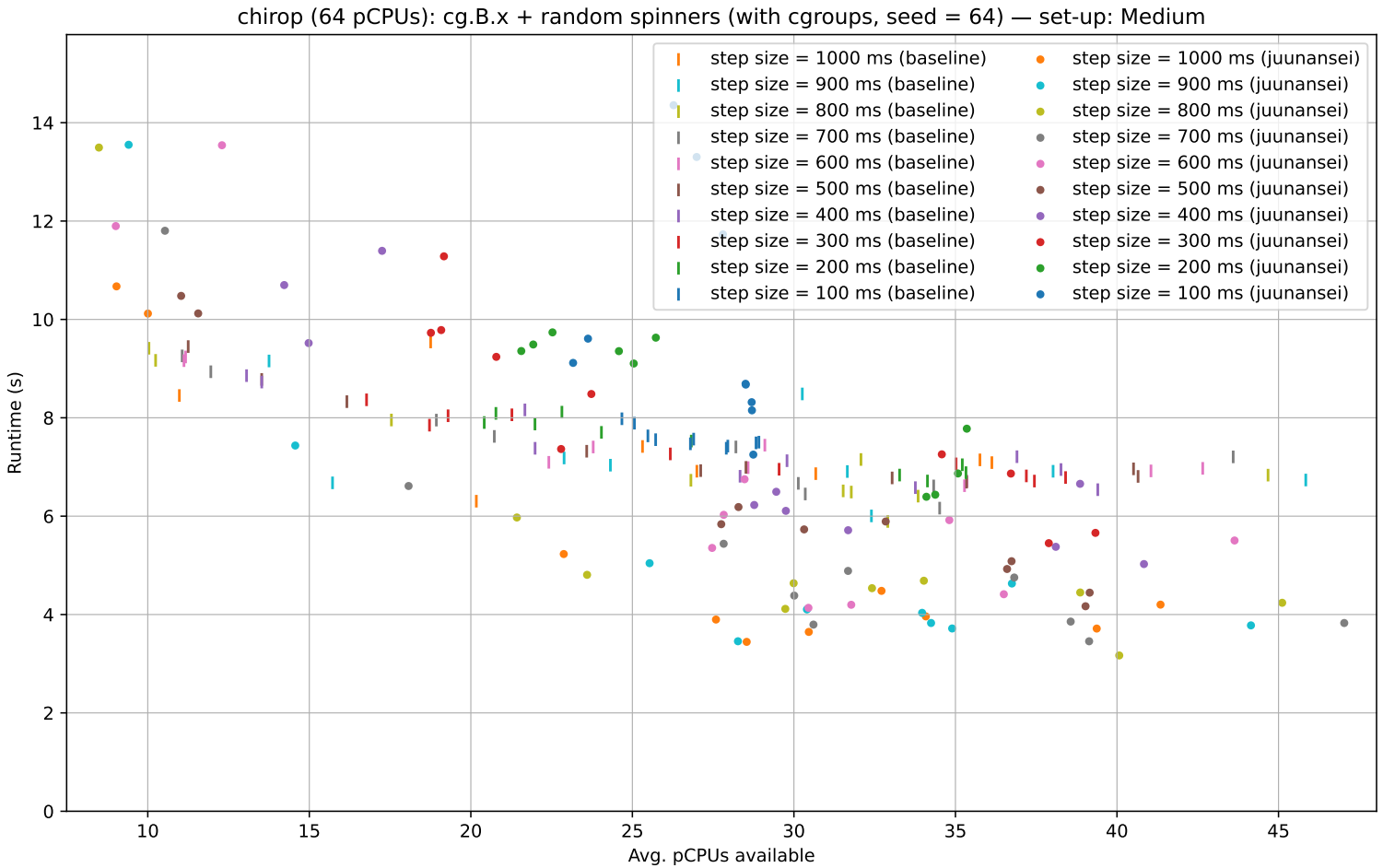
Target OpenMP application: FT

Results: Experiment set-up: Medium [1/2]



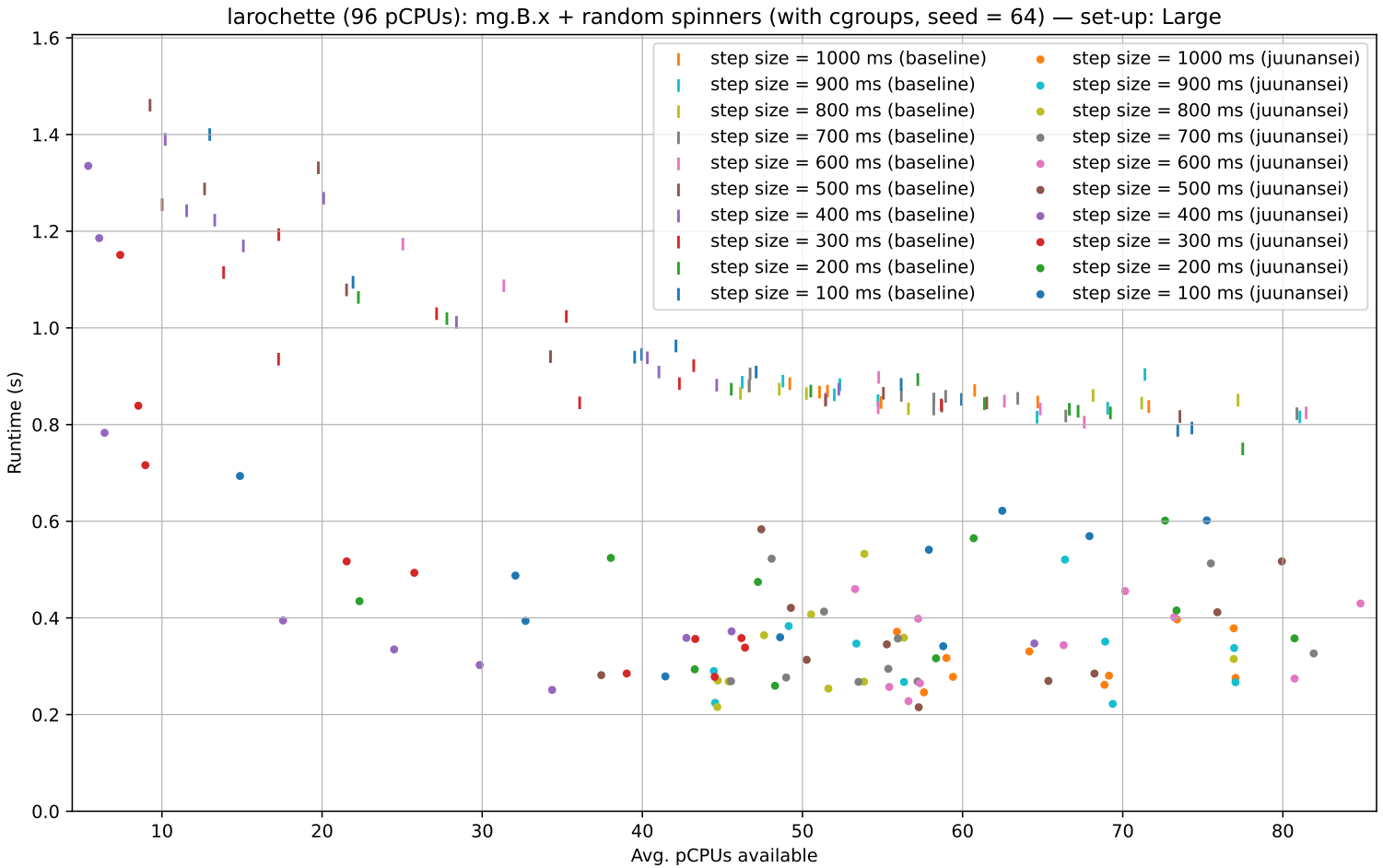
Target OpenMP application: SP

Results: Experiment set-up: Medium [2/2]



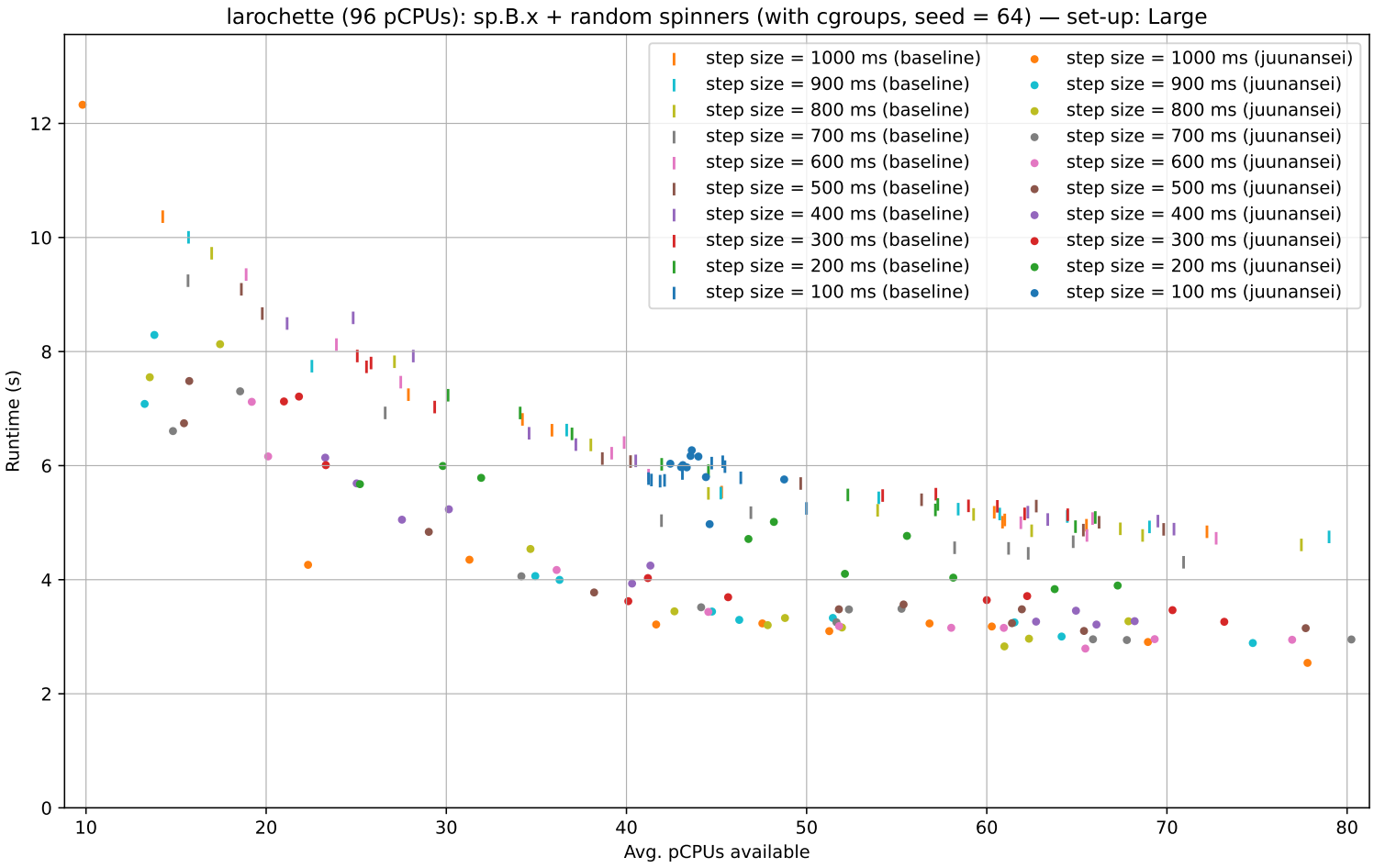
Target OpenMP application: CG

Results: Experiment set-up: Large [1/2]



Target OpenMP application: MG

Results: Experiment set-up: Large [2/2]



Target OpenMP application: SP

Conclusion

Future Research Directions

- Explore broader integration of paravirtualized scheduling interfaces in production hypervisors and OpenMP runtimes.
- Extend phantom-aware techniques to hybrid MPI+OpenMP and other parallel programming models.

Summary

Inria

- OpenMP execution in cloud VMs suffers when guest and host schedulers fall out of sync, especially at barriers.
- Phantom Tracker exposes phantom-induced delays by correlating guest behavior with host scheduling events.
- pv-barrier-sync uses host feedback to dynamically choose spinning vs. blocking at barriers.
- Juunansei extends these insights to adapt OpenMP's degree of parallelization and thread affinity.
- Together, the mechanisms significantly reduce barrier latency and improve performance under oversubscription.

For Follow-up/ Collaboration: himadrics@pm.me